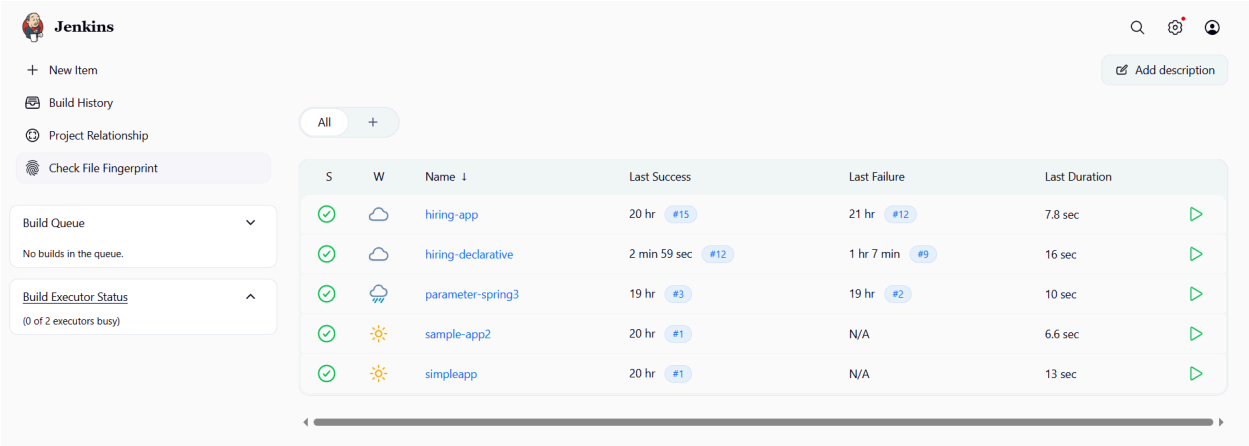
Jenkins challenge

Use the below code and implement our CICD stages.

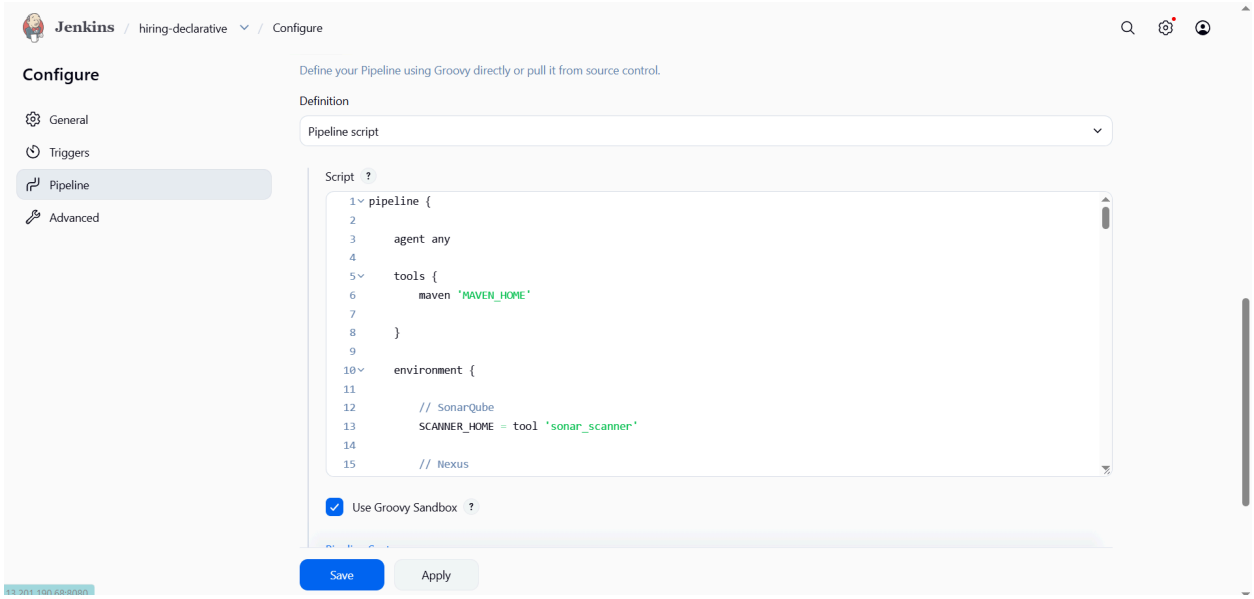
- **Source Code:https://github.com/betawins/hiring-app.git**
  - **Repository contains the application code for the hiring app.**
- **Create one Declarative pipeline job**
- **Create one Scripted pipeline job**
- **Create one multi stage pipeline job**
- **Create one parallel stage pipeline job**

Declarative pipeline job:

Create new item→select pipeline→save



Open job→ click on configure→write pipeline→save& build




```
pipeline {

    agent any

    tools {
        maven 'MAVEN_HOME'

    }

    environment {

        // SonarQube
        SCANNER_HOME = tool 'sonar_scanner'

        // Nexus
        NEXUS_VERSION = "nexus3"
        NEXUS_PROTOCOL = "http"
        NEXUS_URL = "13.232.205.190:8081"
        NEXUS_REPOSITORY = "hiring-app"
        NEXUS_CREDENTIAL_ID = "nexus"

        // Artifact Details
        GROUP_ID = "in.javahome"
```

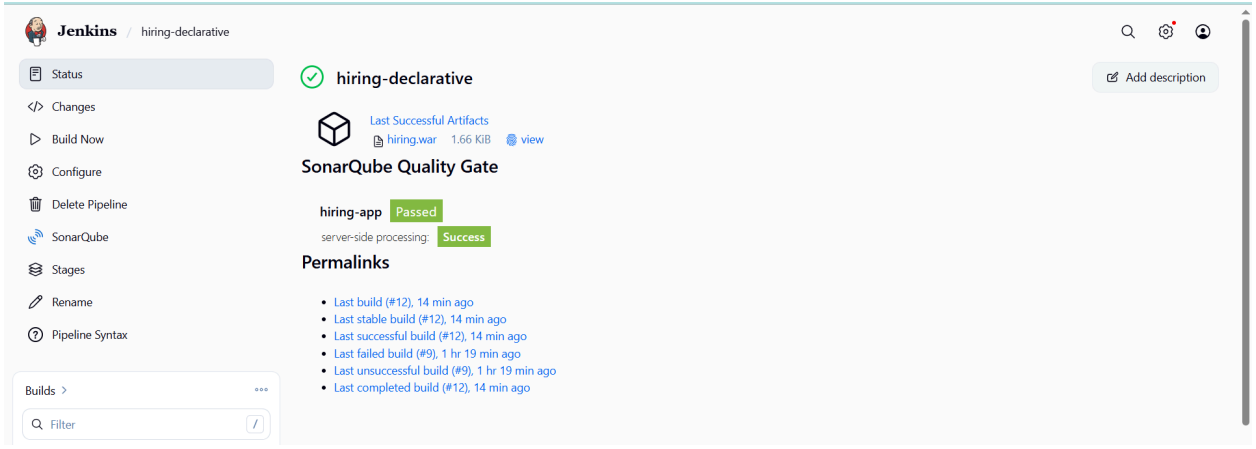
```
    ARTIFACT_ID = "hiring"
    VERSION = "1.0.${BUILD_NUMBER}-SNAPSHOT"

    // Tomcat
    TOMCAT_URL = "http://13.201.30.120:8080/"
    TOMCAT_CREDENTIAL_ID = "tomcat"

    // Slack
    SLACK_CHANNEL = "jobs"
}

stages {

    stage('Clean Workspace') {
        steps {
            cleanWs()
        }
    }

    stage('Clone Source Code') {
        steps {

            git branch: 'main',
            url: 'https://github.com/betawins/hiring-app.git'
        }
    }

    stage('Build Application') {
        steps {

            sh "${tool 'MAVEN_HOME'}/bin/mvn clean install"
        }
    }

    stage('Run Unit Tests') {
        steps {

            sh 'mvn test'
        }
    }
```

```
stage('SonarQube Code Analysis') {

   steps {

      withSonarQubeEnv('sonarqube') {

         sh """
         ${SCANNER_HOME}/bin/sonar-scanner \
         -Dsonar.projectKey=hiring-app \
         -Dsonar.projectName=hiring-app \
         -Dsonar.sources=.
         """
      }
   }
}

stage('Package Application') {

   steps {

      sh 'mvn clean package'
   }
}

stage('Archive Artifact') {

   steps {

      archiveArtifacts artifacts: 'target/hiring.war'
   }
}

stage('Upload WAR to Nexus') {

   steps {

      nexusArtifactUploader(

         nexusVersion: NEXUS_VERSION,
```

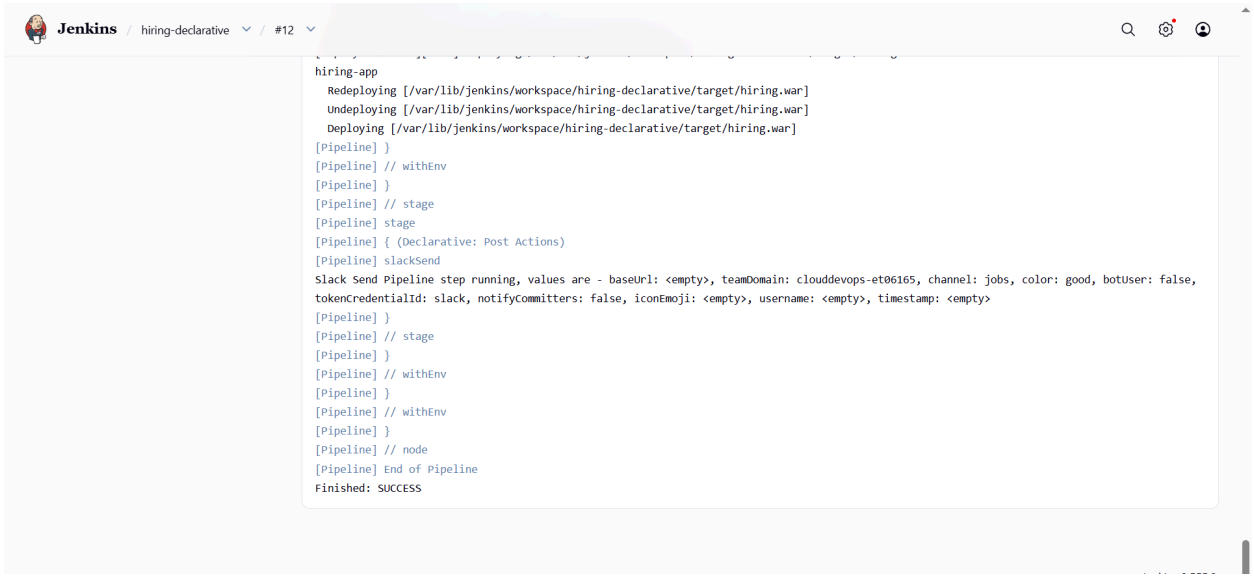
```
            protocol: NEXUS_PROTOCOL,
            nexusUrl: NEXUS_URL,
            repository: NEXUS_REPOSITORY,
            credentialsId: NEXUS_CREDENTIAL_ID,
            groupId: GROUP_ID,
            version: VERSION,

            artifacts: [
               [
                  artifactId: ARTIFACT_ID,
                  classifier: '',
                  file: 'target/hiring.war',
                  type: 'war'
               ]
            ]
         )
      }
   }

   stage('Deploy to Tomcat') {

      steps {

         deploy adapters: [
            tomcat9(
               credentialsId: TOMCAT_CREDENTIAL_ID,
               path: '',
               url: "${TOMCAT_URL}"
            )
         ],

         contextPath: 'hiring-app',
         war: 'target/hiring.war'
      }
   }
}

post {

   success {
```

```
            slackSend(
                channel: "${SLACK_CHANNEL}",
                color: 'good',
                message: """
SUCCESS:
Job Name: ${env.JOB_NAME}
Build Number: ${env.BUILD_NUMBER}
Build URL: ${env.BUILD_URL}
"""
            )
        }

        failure {

            slackSend(
                channel: "${SLACK_CHANNEL}",
                color: 'danger',
                message: """
FAILED:
Job Name: ${env.JOB_NAME}
Build Number: ${env.BUILD_NUMBER}
Build URL: ${env.BUILD_URL}
"""
            )
        }
    }
}
```

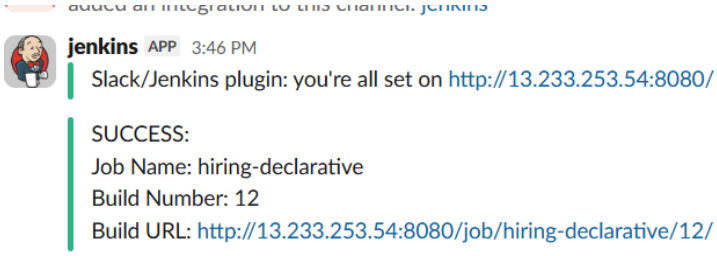
Build successfully



Check console output:

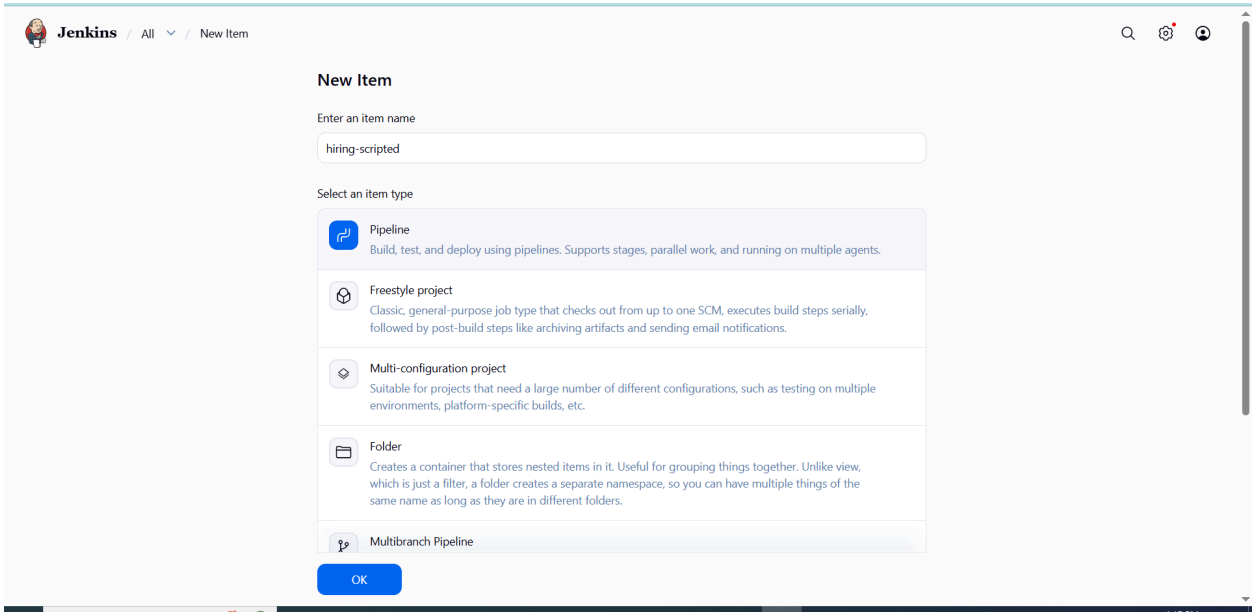


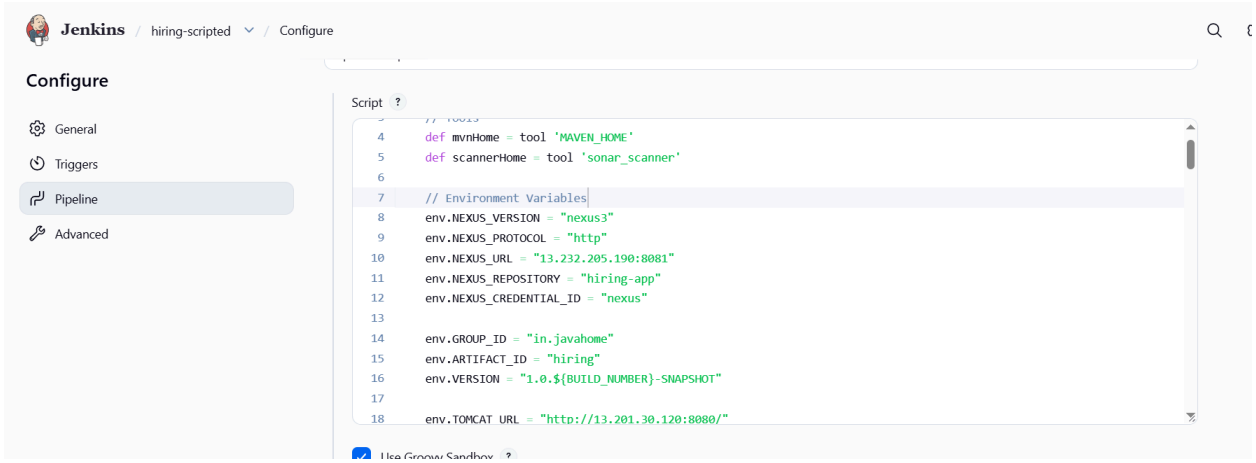
Slack notification



Scripted pipeline job:

Create new item→select pipeline→configure→seclect scripted pipeline






```
node {

   // Tools
   def mvnHome = tool 'MAVEN_HOME'
```

```
def scannerHome = tool 'sonar_scanner'

// Environment Variables
env.NEXUS_VERSION = "nexus3"
env.NEXUS_PROTOCOL = "http"
env.NEXUS_URL = "13.232.205.190:8081"
env.NEXUS_REPOSITORY = "hiring-app"
env.NEXUS_CREDENTIAL_ID = "nexus"

env.GROUP_ID = "in.javahome"
env.ARTIFACT_ID = "hiring"
env.VERSION = "1.0.${BUILD_NUMBER}-SNAPSHOT"

env.TOMCAT_URL = "http://13.201.30.120:8080/"
env.TOMCAT_CREDENTIAL_ID = "tomcat"

env.SLACK_CHANNEL = "jobs"

try {

   stage('Clean Workspace') {

      cleanWs()
   }

   stage('Clone Source Code') {

      git branch: 'main',
      url: 'https://github.com/betawins/hiring-app.git'
   }
```

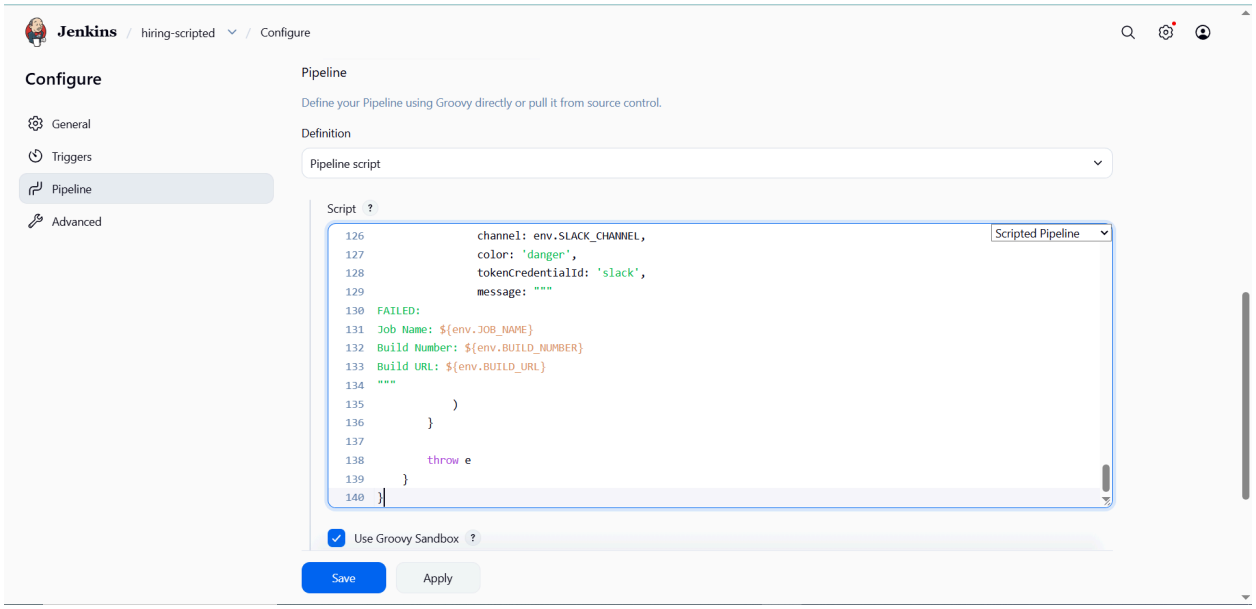
```
stage('Build Application') {

    sh "${mvnHome}/bin/mvn clean install"
}

stage('Run Unit Tests') {

    sh "${mvnHome}/bin/mvn test"
}

stage('SonarQube Code Analysis') {

    withSonarQubeEnv('sonarqube') {

        sh """
        ${scannerHome}/bin/sonar-scanner \
        -Dsonar.projectKey=hiring-app \
        -Dsonar.projectName=hiring-app \
        -Dsonar.sources=.
        """
    }
}

stage('Package Application') {

    sh "${mvnHome}/bin/mvn clean package"
}

stage('Archive Artifact') {

    archiveArtifacts artifacts: 'target/hiring.war'
```

```
}

stage('Upload WAR to Nexus') {

    nexusArtifactUploader(

        nexusVersion: env.NEXUS_VERSION,
        protocol: env.NEXUS_PROTOCOL,
        nexusUrl: env.NEXUS_URL,
        repository: env.NEXUS_REPOSITORY,
        credentialsId: env.NEXUS_CREDENTIAL_ID,
        groupId: env.GROUP_ID,
        version: env.VERSION,

        artifacts: [
            [
                artifactId: env.ARTIFACT_ID,
                classifier: '',
                file: 'target/hiring.war',
                type: 'war'
            ]
        ]
    )
}

stage('Deploy to Tomcat') {

    deploy adapters: [
        tomcat9(
            credentialsId: env.TOMCAT_CREDENTIAL_ID,
            path: '',
```

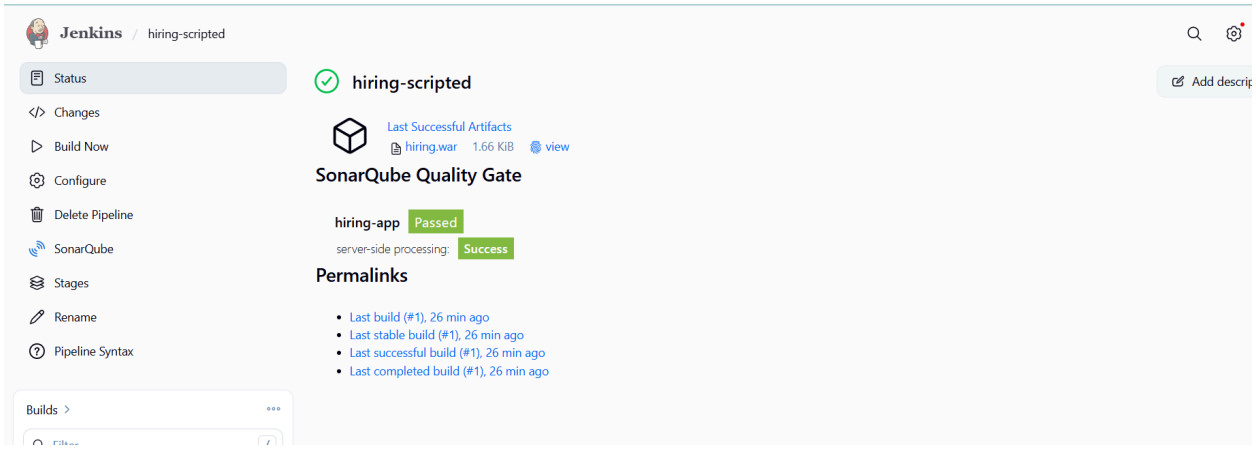
```
                url: env.TOMCAT_URL
            )
        ],

        contextPath: 'hiring-app',
        war: 'target/hiring.war'
    }

    stage('Slack Notification Success') {

        slackSend(
            channel: env.SLACK_CHANNEL,
            color: 'good',
            tokenCredentialId: 'slack',
            message: """
SUCCESS:
Job Name: ${env.JOB_NAME}
Build Number: ${env.BUILD_NUMBER}
Build URL: ${env.BUILD_URL}
"""
        )
    }

  } catch (Exception e) {

    stage('Slack Notification Failure') {

        slackSend(
            channel: env.SLACK_CHANNEL,
            color: 'danger',
            tokenCredentialId: 'slack',
```

```
            message: """
FAILED:
Job Name: ${env.JOB_NAME}
Build Number: ${env.BUILD_NUMBER}
Build URL: ${env.BUILD_URL}
"""
          )
        }

        throw e
    }
}
```

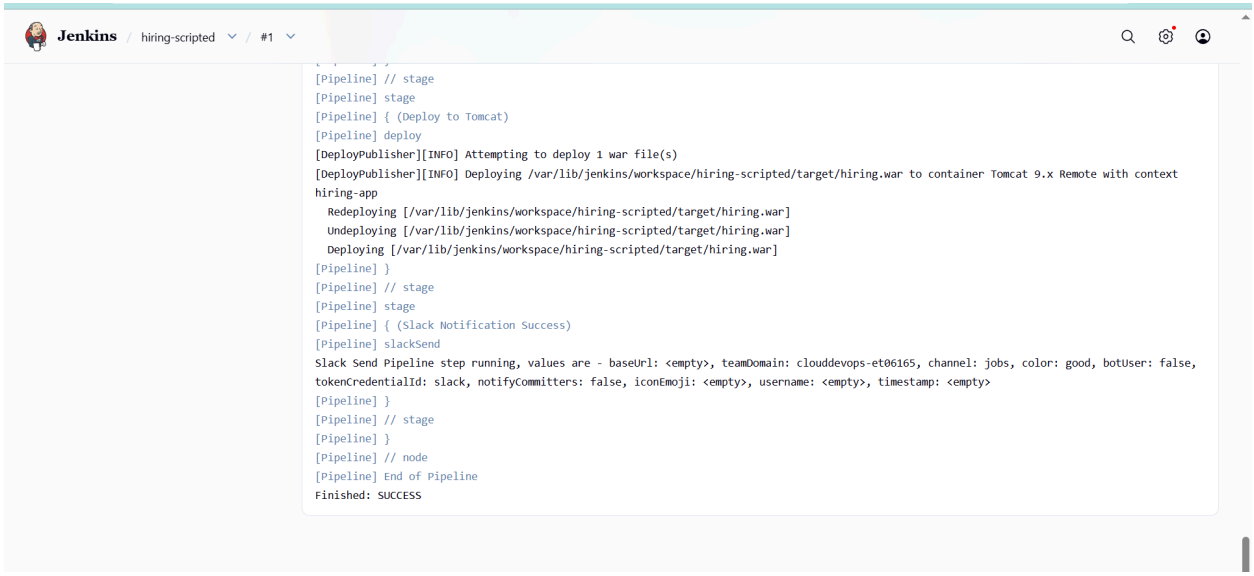
save&build



Build successfull



Check console output



Slack notification

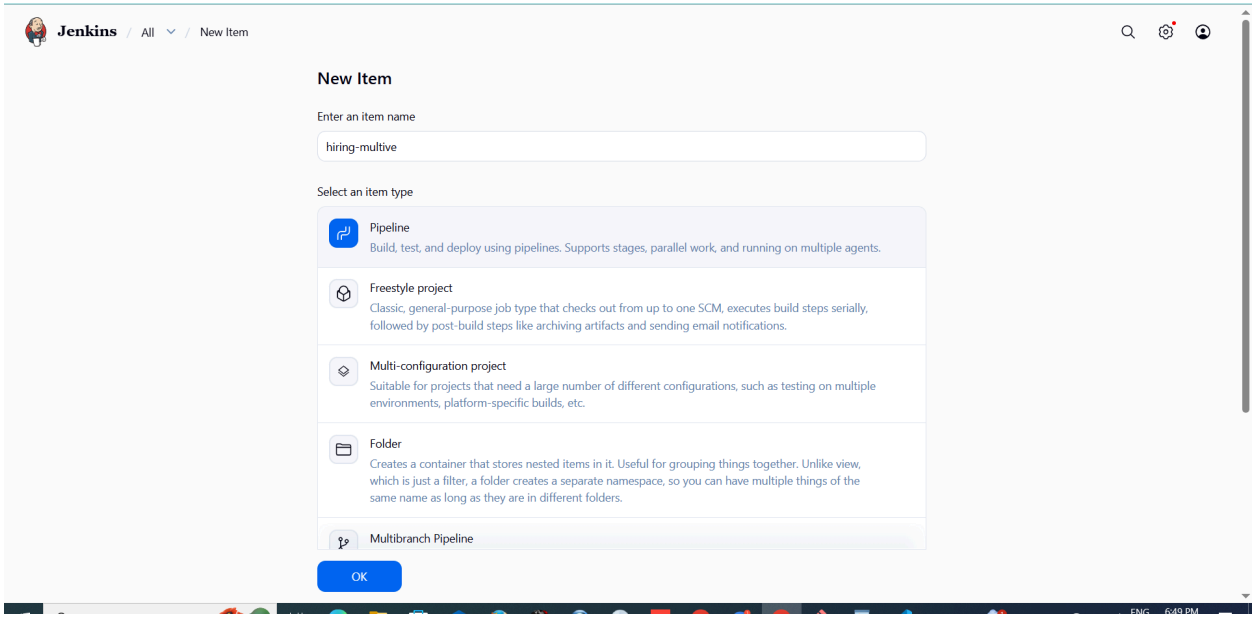
multi stage pipeline job:

here is a proper Multi-Stage Declarative Jenkins Pipeline using:

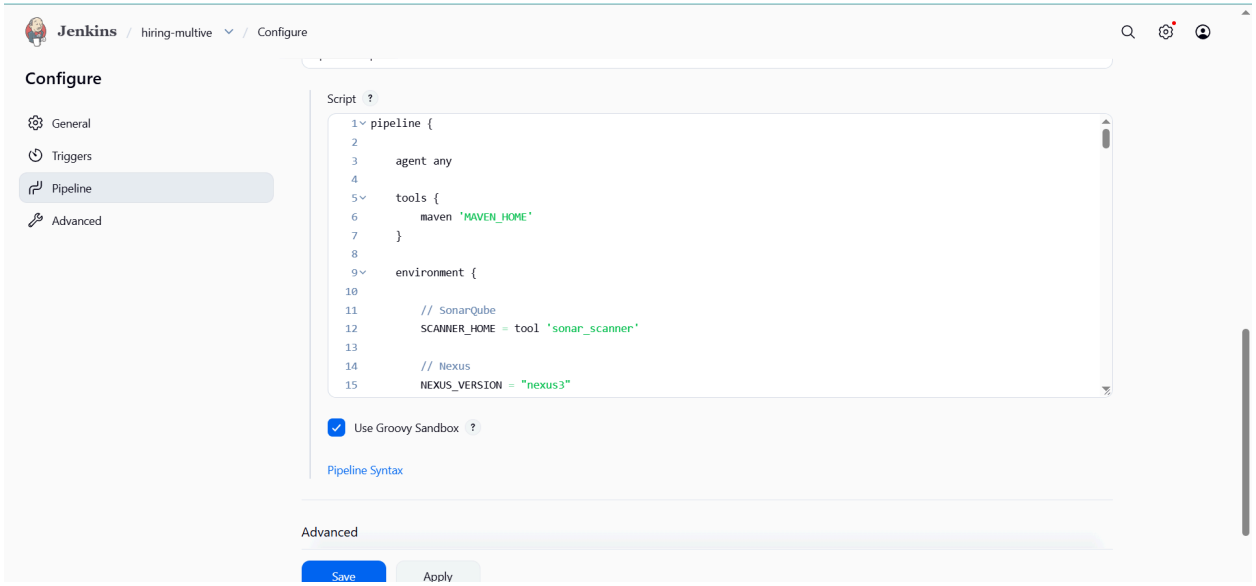
- GitHub
- Maven
- SonarQube
- Nexus
- Tomcat
- Slack

This pipeline separates each CI/CD activity into independent stages.

Create new item→select pipeline→configure



Write pipeline



```
pipeline {

    agent any

    tools {
        maven 'MAVEN_HOME'
    }

    environment {

        // SonarQube
        SCANNER_HOME = tool 'sonar_scanner'

        // Nexus
        NEXUS_VERSION = "nexus3"
        NEXUS_PROTOCOL = "http"
        NEXUS_URL = "13.232.205.190:8081"
        NEXUS_REPOSITORY = "hiring-app"
        NEXUS_CREDENTIAL_ID = "nexus"

        // Artifact Details
        GROUP_ID = "in.javahome"
        ARTIFACT_ID = "hiring"
        VERSION = "1.0.${BUILD_NUMBER}-SNAPSHOT"

        // Tomcat
        TOMCAT_URL = "http://13.201.30.120:8080/"
        TOMCAT_CREDENTIAL_ID = "tomcat"

        // Slack
        SLACK_CHANNEL = "jobs"
```

```
}

stages {

    stage('Stage-1: Clean Workspace') {

        steps {

            cleanWs()
        }
    }

    stage('Stage-2: Clone Source Code') {

        steps {

            git branch: 'main',
            url: 'https://github.com/betawins/hiring-app.git'
        }
    }

    stage('Stage-3: Validate Project') {

        steps {

            sh 'mvn validate'
        }
    }

    stage('Stage-4: Compile Application') {
```

```
    steps {

        sh 'mvn compile'
    }
}

stage('Stage-5: Run Unit Tests') {

    steps {

        sh 'mvn test'
    }
}

stage('Stage-6: SonarQube Analysis') {

    steps {

        withSonarQubeEnv('sonarqube') {

            sh """
            ${SCANNER_HOME}/bin/sonar-scanner \
            -Dsonar.projectKey=hiring-app \
            -Dsonar.projectName=hiring-app \
            -Dsonar.sources=.
            """
        }
    }
}

stage('Stage-7: Package WAR File') {
```

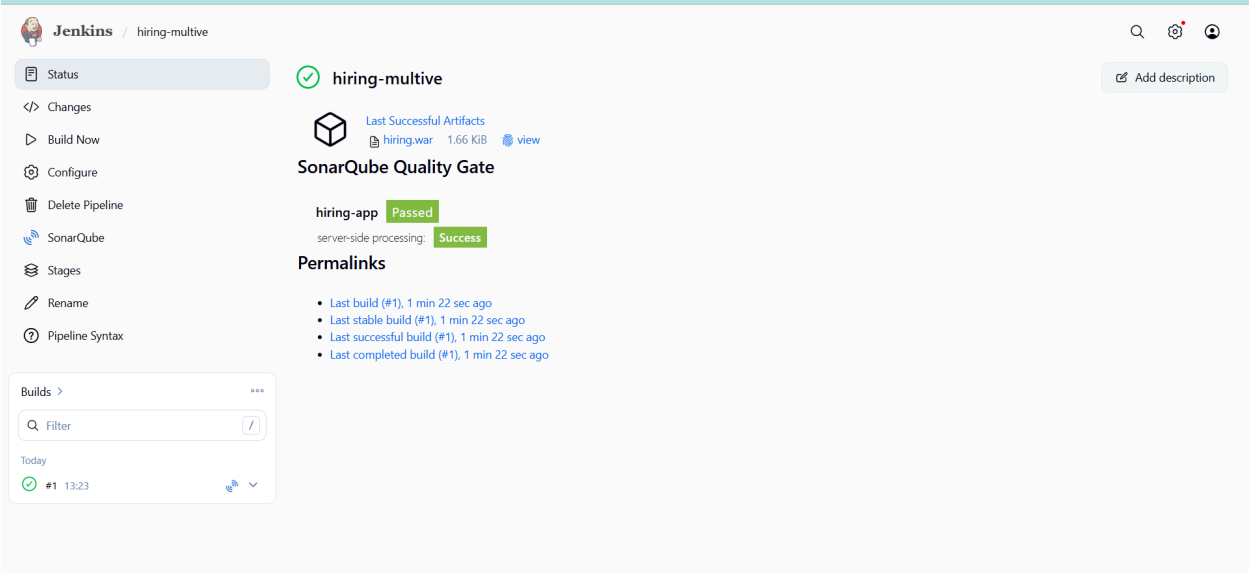
```
    steps {

        sh 'mvn clean package'
    }
}

stage('Stage-8: Archive Artifact') {

    steps {

        archiveArtifacts artifacts: 'target/hiring.war'
    }
}

stage('Stage-9: Upload Artifact to Nexus') {

    steps {

        nexusArtifactUploader(

            nexusVersion: NEXUS_VERSION,
            protocol: NEXUS_PROTOCOL,
            nexusUrl: NEXUS_URL,
            repository: NEXUS_REPOSITORY,
            credentialsId: NEXUS_CREDENTIAL_ID,
            groupId: GROUP_ID,
            version: VERSION,

            artifacts: [
                [
```

```
                artifactId: ARTIFACT_ID,
                classifier: '',
                file: 'target/hiring.war',
                type: 'war'
            ]
        ]
    )
  }
}

stage('Stage-10: Deploy WAR to Tomcat') {

  steps {

    deploy adapters: [
      tomcat9(
        credentialsId: TOMCAT_CREDENTIAL_ID,
        path: '',
        url: '${TOMCAT_URL}'
      )
    ],

    contextPath: 'hiring-app',
    war: 'target/hiring.war'
  }
}

stage('Stage-11: Verify Deployment') {

  steps {
```

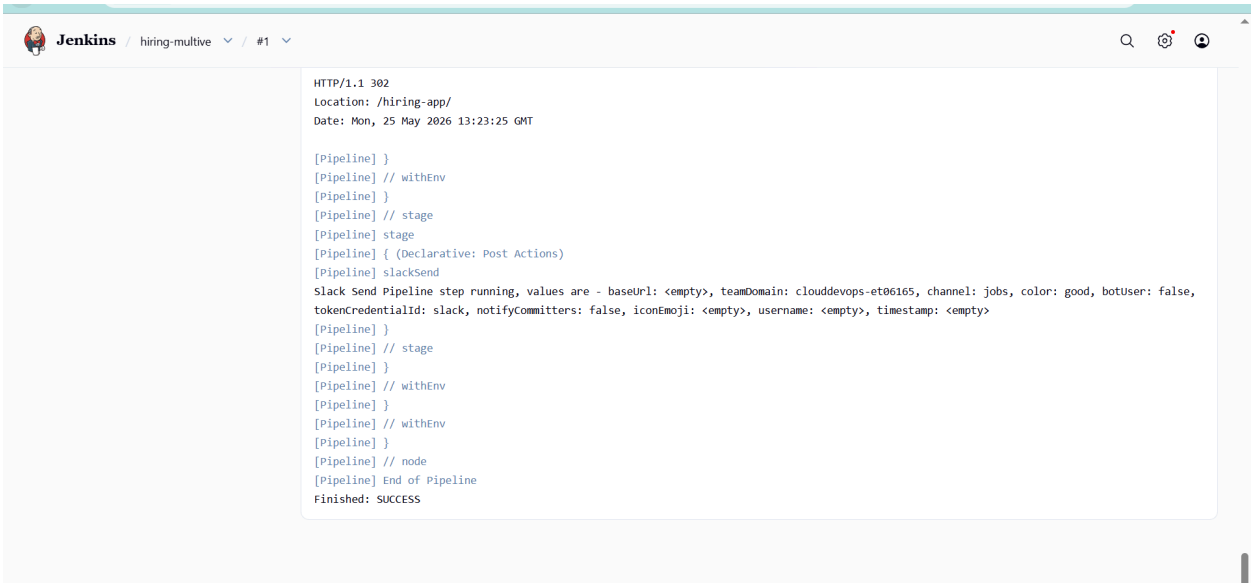
```
            sh """
            curl -I http://13.201.30.120:8080/hiring-app
            """
        }
    }
}

post {

    success {

        slackSend(
            channel: "${SLACK_CHANNEL}",
            color: 'good',
            tokenCredentialId: 'slack',
            message: """
SUCCESS:
Multi-Stage Pipeline Completed Successfully

Job Name: ${env.JOB_NAME}
Build Number: ${env.BUILD_NUMBER}
Build URL: ${env.BUILD_URL}
"""
        )
    }

    failure {

        slackSend(
            channel: "${SLACK_CHANNEL}",
            color: 'danger',
```

```
            tokenCredentialId: 'slack',
            message: """
FAILED:
Multi-Stage Pipeline Failed

Job Name: ${env.JOB_NAME}
Build Number: ${env.BUILD_NUMBER}
Build URL: ${env.BUILD_URL}
"""
        )
    }
  }
}
```

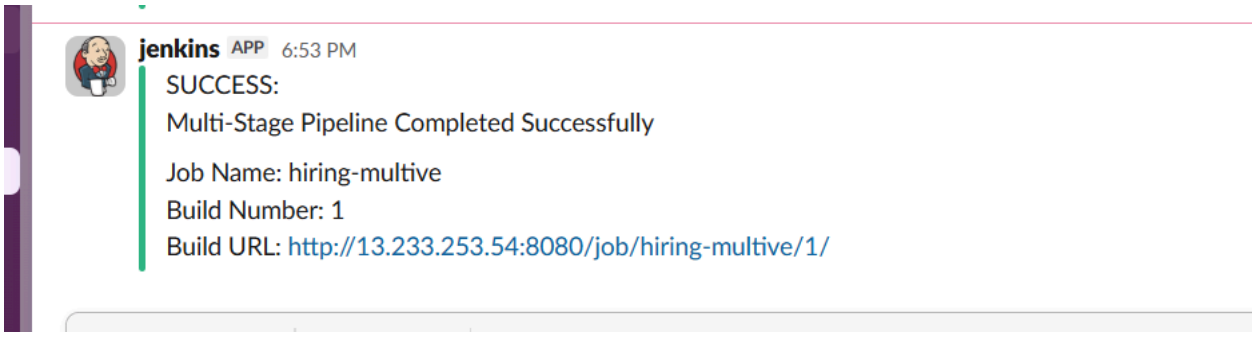
save& build
Build successfull



Console output



Slack notification



parallel stage pipeline job:

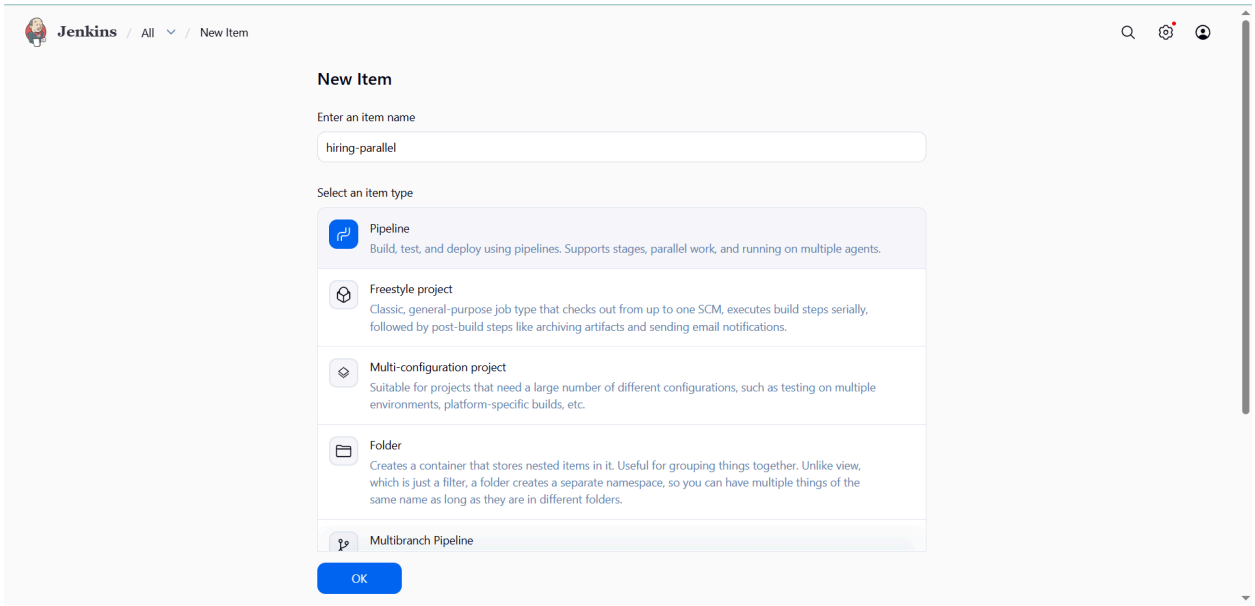
This pipeline runs multiple stages simultaneously to reduce build time.

The following stages run together:

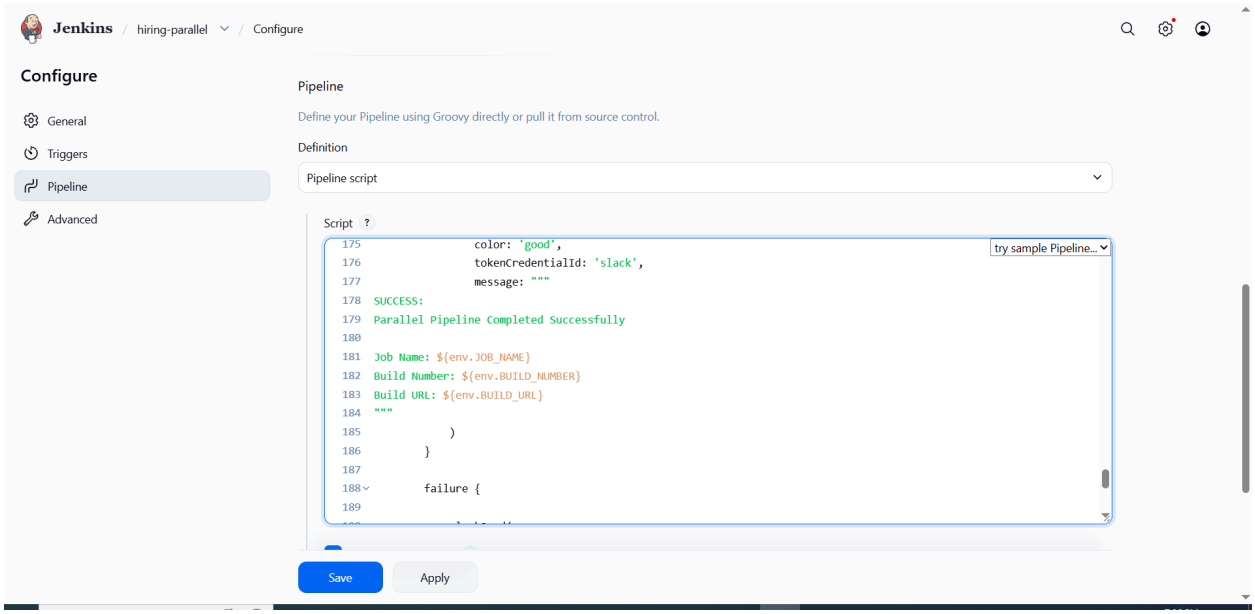
- **Unit Tests**
- **SonarQube Analysis**
- **Security/Code Validation**

This improves CI/CD performance.

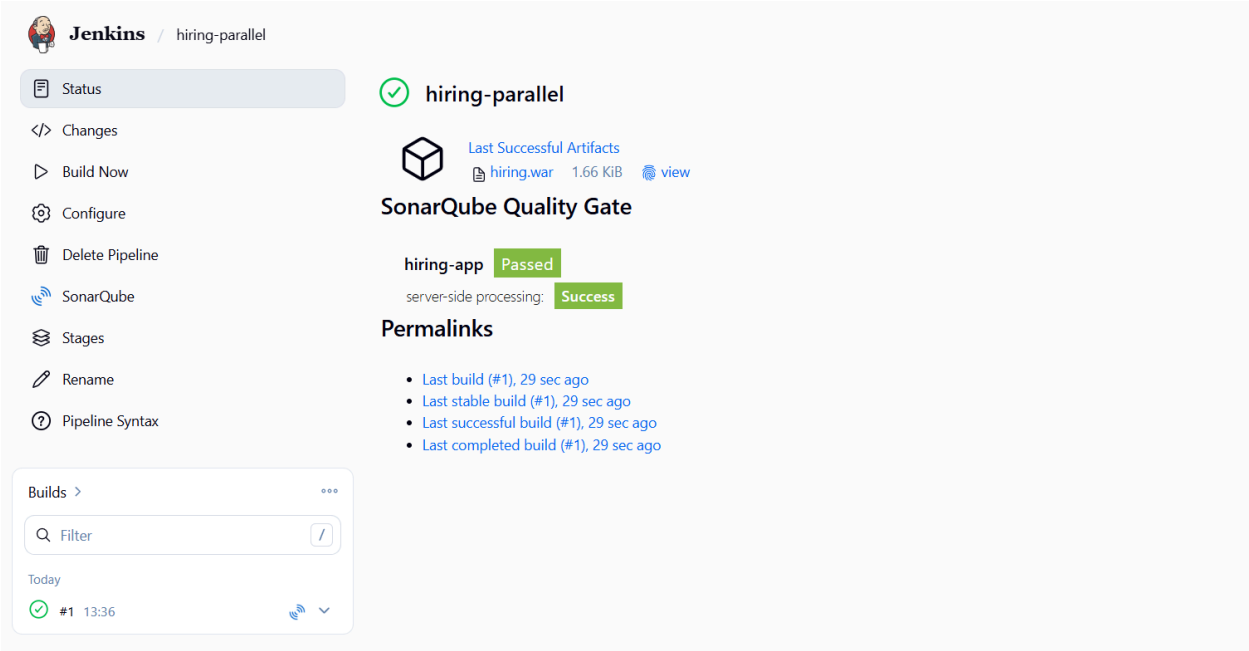
Create new item→select pipeline→confugure



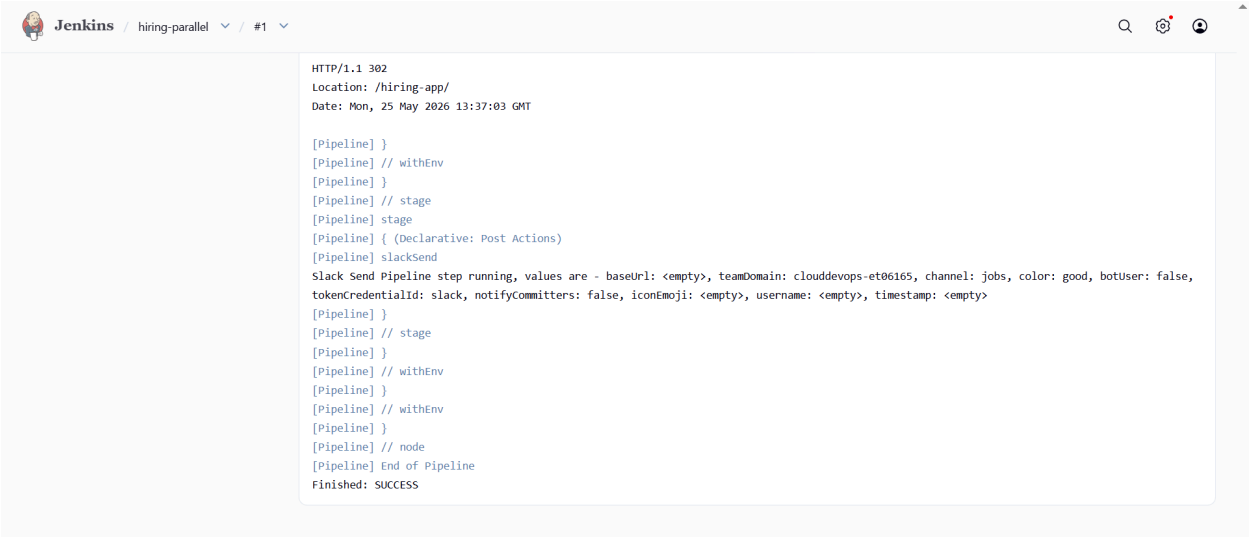
Write pipeline



save&build



Console output



Slack notification